Null Models and Sampling-Based Evaluation of Reinforcement Learning for LLVM Pass Ordering

Dimitrije Pešić
School of Electrical Engineering, University of Belgrade
Belgrade, Serbia
dimitrije.pesicc@gmail.com

***Abstract*—Reinforcement learning (RL) for compiler pass ordering is commonly evaluated by comparing a deterministic policy rollout against a fixed optimization level such as `-O3`. We show that both choices can reverse conclusions. We introduce two null models in the same curated 36-pass action space the policies act in: a single random episode, and best-of-N random search. Policies are evaluated both by a deterministic rollout and by best-of-k sampling. Across 880 programs from eleven independent sources, random search in the curated space alone beats `-Oz` on all nine real-code sources (0.9–24.5%), while a graph-neural policy trained on only four small programs beats the budget-matched random null on 33 of 36 suite–seed combinations, significant within ten of eleven sources (one-sided Wilcoxon); closing that margin costs random search 1.4–6$\times$ the compilation budget. Linux kernel code, where random search alone already suffices, is the measured boundary. The same policy matches greedy search at 73% of its compilation budget in all three seeds; an identically trained flat-feature policy does so in one seed of three. Best-of-8 sequences from the curated space, learned or random, yield `.text` sections 10.6% (x86) and 15% (Cortex-M) smaller than `-Oz` and apply faster than the `-Oz` pipeline. Finally, RL gradients alone fail to train the graph encoder; distilling hand-crafted features into it before RL removes this failure. In this setting, evaluation choices changed the conclusions more than the choice of representation did.**

***Index Terms*—compiler optimization, LLVM, phase ordering, reinforcement learning, graph neural networks, evaluation methodology**

I. Introduction

The order in which a compiler applies its optimization passes changes the quality of the generated code, and choosing a good order per program, the *phase-ordering problem*, is undecidable in general [1], and exhaustive enumeration is feasible only with heavy pruning [2]. Fixed pipelines such as LLVM's `-O3` and `-Oz` apply one hand-tuned sequence to every program, which makes program-specific orderings an attractive target for machine learning; a line of work applies deep reinforcement learning to the problem [3], [4], [5], [6].

Reported results, however, rest on two rarely questioned evaluation conventions. First, learned policies are compared against fixed optimization levels (often `-O3`, a *speed*-oriented pipeline, even when the objective is code *size*) rather than against a null model measuring how much of the gain needs no learning. Second, policies are evaluated by a single deterministic (argmax) rollout: the mode of the policy distribution, not the distribution.

This paper asks what remains of an RL pass-ordering system once both conventions are replaced. We train proximal policy optimization (PPO) [7] agents with two state representations, the flat 56-dimensional Autophase feature vector [3] and a custom GraphSAGE [8] encoder over control- and data-flow graphs parsed from LLVM IR, under a controlled protocol in which the representation is the only variable, and we evaluate them on 880 programs drawn from eleven independent sources against null models defined in the same action space and with the same step budget.

Our contributions are threefold. (1) We show that a curated 36-pass action space is itself the dominant effect: best-of-50 random search inside it comes within 0.5% of greedy search and beats `-Oz` on every real-code suite, a fact any evaluation should control for. (2) The evaluation mode reverses the ranking of representations: under argmax the two agents are indistinguishable, while under best-of-k sampling the graph policy beats the paired null on 33 of 36 suite–seed combinations (24 of 24 on the original seven sources) and matches greedy-search quality at 73% of its budget in every seed, which the flat policy does in one seed of three. (3) We show that RL gradients alone fail to train the graph encoder, and that distilling Autophase features into the encoder before RL removes the failure, improving deterministic-rollout generalization by about 9%. All results are reproducible from a public repository with pinned dependencies.[1]

II. Related Work

AutoPhase [3] introduced deep RL over a flat feature vector for phase ordering; CompilerGym [4] standardized the environment we build on. ProGraML [9] showed that graph representations of programs outperform flat features on supervised compiler tasks, and MLGO [10] deployed learned policies inside production LLVM for inlining. POSET-RL [5] and the coreset approach of Liang et al. [6] reduce the action space before learning; the latter, closest to ours, selects from a 50-sequence coreset with a GNN and reports an average 4.7% gain over `-Oz`. Our study is complementary: it measures *where such gains come from*, with null models inside the same reduced space, a control absent from prior evaluations, and shows that the decoding mode changes conclusions. Cummins et al. [4] report that plain random search is competitive with

[1] `https://github.com/dimitrijepesic/compiler-opt`

several sophisticated autotuners on cBench; we formalize that observation into a null-model protocol and replicate it across nine real-code sources. Mammadli et al. [11] train RL over `-O3` sub-sequences and find it inferior to `-O3` on held-out programs, consistent with our argmax findings. Large language models have also been applied, at a far larger budget [12].

III. Experimental Protocol

Environment and metric. We use CompilerGym 0.2.5 (`llvm-ic-v0`, LLVM 10) [4]. The optimization objective is LLVM IR instruction count (IC); Section IV-E validates IC against actual binary section sizes. `-O3`/`-Oz` baselines are the environment's real optimization-level observations, not approximated pass lists.

Action space. Profiling all 124 available passes on the 14-program cBench training split yields 36 passes that each improve at least one program; these cover 100% of the observed single-pass improvement. All agents and null models operate in this space with 45-step episodes. The reference greedy search tries all 124 passes at every step and stops when none improves ($\approx$1 970 compilations per cBench program).

Agents and training. Both agents use PPO (clip 0.2, γ=0.99, GAE λ=0.95) with truncation-aware advantage bootstrapping, KL-based early stopping of update epochs (target 0.02), and a linearly decaying entropy coefficient. The Autophase agent observes a $\log(1+x)$-transformed 56-dimensional feature vector; the GNN agent parses the IR into a graph (one-hot opcode plus four scalar features per instruction; separate control-flow and data-flow edge types) and encodes it with a three-layer edge-type-aware GraphSAGE network. Both train on the *same* four small cBench programs (<3000 instructions), for the same 100 000 environment steps, with three seeds each; checkpoints are selected on a three-program validation subset. Apart from the joint optimizer that the shared trainable encoder requires (encoder learning rate 10^{-4}), the representation is the only variable.

Null models. Two null models are measured in the same 36-pass space: (a) a single random 45-step episode (the null for one policy rollout) and (b) the best of N random episodes (the null for search). When a policy is evaluated by best-of-k sampling, its null is the best of the *same number* k of stored random episodes on the same program, paired by construction.

Evaluation. Policies are evaluated in two modes: a single deterministic argmax rollout, and best-of-k stochastic rollouts sampled from the policy distribution (GNN rollouts sample with the encoder's dropout active, which adds exploration noise). The evaluation battery spans eleven sources: cBench (val+test, 9), MiBench [13] (40), CHStone [14] (12), NPB (122), BLAS (50), a 97-program POJ-104 sample, 150-program samples of the AnghaBench, GitHub and Linux-kernel datasets shipped with CompilerGym [4], and two synthetic generators (csmith [15], 50; llvm-stress, 50); in total 871 battery programs plus cBench, none seen in training. Policy evaluation uses programs up to 6000 instructions (831 eligible; eight fail feature extraction, leaving 823) to bound rollout cost. The four added sources store 16 samples per program: the first eight for comparability, the disjoint second eight as a built-in resampling check.

TABLE I
Total IR instruction count per suite: real optimization levels vs. best-of-50 random episodes in the curated 36-pass space.

Suite (n)	O0	-O3	-Oz	Rnd-50	ΔOz
AnghaBench (150)	8 902	4 193	4 043	**4 006**	+0.9%
GitHub (150)	224 490	222 369	224 171	**221 297**	+1.3%
MiBench (40)	12 270	6 317	4 841	**4 765**	+1.6%
Linux (150)	255 140	250 862	252 031	**246 983**	+2.0%
BLAS (50)	40 745	39 141	37 838	**36 757**	+2.9%
cBench (9)	232 442	163 834	115 987	**111 006**	+4.3%
CHStone (12)	19 857	14 182	9 834	**9 097**	+7.5%
POJ-104 (97)	21 099	11 083	8 334	**7 622**	+8.5%
NPB (122)	130 327	70 659	59 542	**44 981**	+24.5%
csmith (50)	352 963	82 950	**57 625**	60 626	−5.2%
llvm-stress (50)	5 739	238	**234**	238	−1.7%

Encoder pretraining. In one arm, the GNN encoder is first trained to regress the Autophase vector from the program graph (2430 states collected by random episodes on the training programs) and then fine-tuned with RL.

IV. Results

A. The curated action space, not learning, is the main effect

Table I compares real optimization levels against best-of-50 random episodes in the 36-pass space. Random search beats `-Oz` on all nine real-code sources, by 0.9% (AnghaBench) up to 24.5% (NPB); on NPB the summed IC of even *single* random episodes is 12% below `-Oz`, a total driven by the largest programs (only 32% of individual episodes win). The margins are thinnest on AnghaBench, GitHub and Linux (0.9–2.0%), whose code largely arrives pre-optimized (on GitHub `-Oz` removes only 0.14% of O0 IC; on both, `-O3` beats `-Oz`). Two boundaries are equally consistent: `-O3` is the wrong yardstick for size (on 8% of the programs under 500 instructions it *increases* IC above `-O0`), and on the synthetic generators the full `-Oz` pipeline keeps its advantage (csmith 5.2%, llvm-stress 1.7%): the curated space was distilled from real code and transfers across real code only. Learned-policy results must therefore be read against these nulls.

B. Sampling reverses the ranking of representations

Under argmax, both policies transfer poorly to programs one to two orders of magnitude larger than the training set (the GNN's deterministic mode degenerates to 79 686 total IC on NPB vs. `-Oz` 53 085), and the representations are statistically indistinguishable. Sampling the same checkpoints changes the picture. Table II reports best-of-8 sampled rollouts against the paired best-of-8 random null: the GNN policy beats its null on 33 of 36 suite–seed totals across the eleven sources, 24 of 24 on the original seven. Since seeds within a suite share the stored null episodes, the primary test uses the twelve independent suite/split units: the GNN wins ten under every seed (exact sign test p=0.019; the original eight alone give p=3.9$\times10^{-3}$), and one-sided Wilcoxon tests are significant within ten of eleven sources (largest p=0.025 on MiBench). The exception is Linux kernel code: the random null itself

TABLE II

BEST-OF-8 SAMPLED ROLLOUTS (PER-CELL MEDIAN OVER THREE SEEDS) VS. THE PAIRED BEST-OF-8 RANDOM NULL AND -OZ. POLICIES WERE TRAINED ON FOUR SMALL CBENCH PROGRAMS ONLY. ISO-k: MEDIAN RANDOM EPISODES TO MATCH THE GNN BEST-OF-8; SAFE%: MEAN PER-PROGRAM GAIN OF min(BEST-OF-8, `-Oz`) OVER `-Oz` (MEDIAN SEED). ADDED SOURCES USE THE FIRST 8 OF 16 STORED SAMPLES; n COUNTS PROGRAMS WHERE ALL SEEDS SUCCEEDED.

Suite (n)	Null	-Oz	AP	GNN	iso-k	safe%
MiBench (40)	4 828	4 841	4 783	**4 782**	23	1.8
CHStone (12)	9 241	9 834	9 133	**9 120**	32	7.3
BLAS (50)	37 099	37 838	36 998	**36 847**	20	2.0
csmith (28)	17 555	20 226	17 179	**17 064**	>50	18.2
NPB (120)	40 301	53 085	40 822	**39 909**	11	10.0
POJ-104 (97)	7 853	8 334	7 917	**7 730**	12	7.9
AnghaB. (150)	4 049	4 043	4 052	**4 027**	21	1.2
GitHub (140)	79 713	80 661	79 738	**79 676**	27	1.2
Linux (136)	**147 216**	148 029	147 278	147 378	3	0.6
llvm-str. (50)	255	**234**	245	240	49	3.1

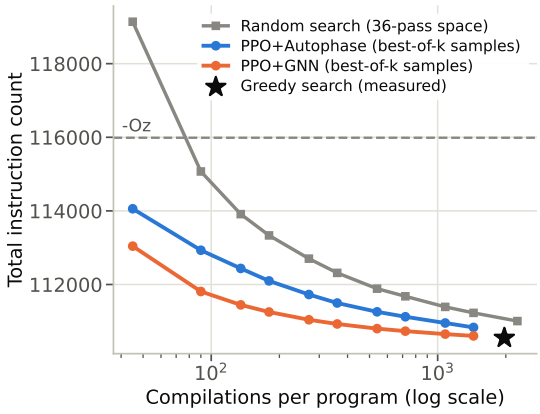

Fig. 1. Quality vs. budget on cBench val+test: best-of-k sampling (k=1..32, mean over three seeds) vs. expected best-of-k random search and measured greedy search.

beats `-Oz` there by 2%, and the policy adds nothing (per-program $p \geq 0.24$ in two of three seeds; zero of three second-slice wins). GitHub mirrors NPB: one seed loses the summed IC by 17 while the per-program test is significant under every seed ($p \leq 2 \times 10^{-3}$). The identically trained Autophase policy wins 21 of 36 pairs and is significant on five of eleven sources. The margin over the null is modest (0.7–2.8% of suite totals on the original battery) but systematic, and it holds on the synthetic csmith programs where pure random search *loses* to `-Oz`: there the sampled GNN policy is 16% below `-Oz`. Read as *search*, the same margin is large: on eight of the ten sources where the GNN wins, matching its eight samples takes random search a median of 11–49 episodes (1.4–6× the budget); on the other two, cBench and csmith, no seed and one of three seeds respectively are matched within the 50 stored episodes. On Linux, where the policy does not win, three random episodes suffice (Table II, iso-k). Per program on the original battery, the GNN sampler is at least as good as its null on 88–92% of the 347 programs (175W/145T/27L, seed 42), and because the $(k{+}1)$-th candidate can be `-Oz` itself at no extra cost, the deployed sampler is never worse than `-Oz` by construction while averaging 0.6–18.2% per-program savings (Table II, safe%). The policy thus transfers to ten of eleven sources, but only sampling reveals it.

Robustness. A k=32 rerun of the original suites reproduces the suite-total win everywhere except NPB (one seed of three; the total flips in 7 of 12 disjoint eight-sample slices), yet the per-program Wilcoxon on NPB stays significant in every resampling ($p<0.05$ in all twelve slices, $p<10^{-3}$ in each rerun's first eight). Where totals are dominated by a few large programs (NPB, GitHub, Linux), per-suite tests, not win counts, are the primary evidence.

Fig. 1 places these results on a quality-vs.-budget front on cBench. At k=8 (360 compilations per program) the sampled GNN policy is within 0.4% of greedy search (0.06–0.35% across seeds); at k=32 (73% of greedy's budget) it reaches greedy quality, nominally surpassing it in two of three seeds (110 494 and 110 527 vs. 110 550), while all three seeds beat the paired null (111 023). Autophase at k=32 surpasses greedy in one seed (110 338) but trails the null in the other two (111 056, 111 108): the graph policy's advantage is consistency, not peak quality. Within the measured budgets random search never catches the sampler on cBench; on the battery it does so at the larger measured budget (k=32), first on NPB, precisely the suite where random search is strongest. The learned sampler thus reduces the cost of search where search is most expensive.

C. Controls: where the sampling advantage lives

Four controls on the same 347 programs and nulls (seed 42; Table III) decompose the margin. An *untrained* policy with identical architecture and sampling protocol is statistically indistinguishable from random search on every suite (all one-sided $p \geq 0.12$) and loses five of six suite totals: the advantage is learned. Disabling dropout reproduces the result on all six suites (largest p=0.016) and is indistinguishable from dropout-active sampling (58W/226T/63L): the exploration comes from the policy distribution itself. An *open-loop* variant sampling all 45 actions from $\pi(\cdot \mid s_0)$ (one graph pass; best-of-8 then costs eight sequence applications, ≈0.13 s per module) retains 90–141% of the closed-loop gain on MiBench, BLAS and CHStone but loses it on NPB and POJ-104 (overall 0.35): the learned per-program *prior* suffices on simpler suites; state feedback pays off on harder ones. Finally, a fixed eight-sequence *portfolio* mined by greedy set cover from random search on the 14 training programs (no model at inference) beats the null on five of six suites and the learned sampler on MiBench, BLAS and CHStone (one-sided $p \leq 0.006$); the policy keeps better totals on csmith, NPB and POJ-104, on csmith significantly (one-sided p=0.004 on per-program medians across seeds). Much of the sampler's value on small, regular programs is thus distillable into a static sequence list, in direct support of coreset-style approaches [6], while per-program adaptation pays off where random search is weakest.

D. Pretraining unlocks the encoder

All three from-scratch GNN seeds reach the same validation plateau (689) within the first 10–17% of the budget and

TABLE III
CONTROLS (SEED 42, 347 PROGRAMS): SUMMED IC OF BEST-OF-8 VARIANTS VS. THE STORED BEST-OF-8 NULL (116 877); WINS = SUITES (OF 6) BEATING IT. PORTFOLIO-16 IS SCORED AGAINST THE BEST-OF-16 NULL (115 158).

Variant	Wins	Total IC
Untrained policy, best-of-8	1/6	116 941
Open-loop $\pi(\cdot\|s_0)$, best-of-8	4/6	116 377
Portfolio-8 (no model)	5/6	115 972
GNN best-of-8 (no dropout: 115 761)	6/6	115 451
Portfolio-16 (no model, 2× budget)	6/6	114 407

never leave it; the plateau is *worse* than `-Oz` (643) and a single random episode (640): RL gradients alone do not teach the encoder useful features. Distilling Autophase into the encoder before RL (about an hour of CPU) removes the failure in two of two seeds (best validation 651 and 668) and improves the mean full-split argmax totals from 64 550 to 57 980 (validation) and 69 180 to 63 229 (test), 10.2% and 8.6% (9.4% on average), with the better seed within 0.8% of `-Oz` on both splits; it is the first deterministic policy here to beat the single-episode random null on both. Pretraining repairs the policy's *mode*; the from-scratch policy remains the better *sampler*. The two fixes are complementary: pretrain when only one rollout is affordable, sample when eight are.

E. Binary-level metrics

IC is a proxy, so we compiled the optimized modules with the environment's own LLVM 10 code generator and measured section sizes. On programs with at least 1000 instructions, relative IC reduction strongly predicts relative `.text` reduction (Pearson r=0.78, p=2.2×10^{-6}; n=26 points from 13 programs); below that size, fixed code-generation overheads dominate and the proxy weakens. The IC gains translate into bytes on the original battery: across its 347 paired programs, best-of-8 GNN sampling produces `.text` sections 10.6% smaller than `-Oz` (864 887 vs. 967 878 bytes; the random null achieves 10.4%), and across all 371 random search alone is 4.0% below `-Oz`. The 45-pass sequences also apply *faster* than the `-Oz` pipeline (median 15.0–15.8 ms vs. 19.5 ms per module via the plain `opt` CLI; the sequences work outside the RL environment). Cross-compiling the same modules for a Cortex-M-class embedded target (`thumbv7m`, LLVM 10's ARM backend; an approximation, as the IR carries the x86 host datalayout) makes the savings *larger*: best-of-8 sequences produce `.text` sections 14.9–15.0% below `-Oz` (839 vs. 986 kB over the same 347 programs), the sampler and the random null now equal: on this target the curated space delivers the entire margin. Total footprint is dominated (>99.9%) by data and BSS sections, invariant under pass ordering (<0.03%), so `.text` is the informative metric.

V. DISCUSSION AND LIMITATIONS

Three practices follow from these results. Evaluations of learned pass ordering should (1) report a random null inside the same action space and budget; (2) report sampling-based in addition to deterministic evaluation, since the two can reverse rankings; and (3) never use `-O3` as a size baseline. Limitations: the study optimizes IR instruction count with a validated but size-bounded correlation to binary sections; training used four small programs by design (the experimental control), and a single-seed mixed-size arm did not improve generalization; runtime is not measured; the stack is CompilerGym 0.2.5 (LLVM 10); and the GNN costs ∼19× more per training step, though open-loop deployment cuts inference to one graph pass.

VI. CONCLUSION

Under controlled measurement, the value of RL for LLVM pass ordering redistributes: most of the headline gain over `-Oz` belongs to the curated action space; the contribution of the learned graph policy that survives the controls is that of an *amortized searcher*: greedy-search quality at a fraction of its compilation budget, transferable from four small programs to ten of eleven program sources. Its encoder trains only after a cheap pretraining step. Throughout, evaluation choices changed the conclusions more than representation choices did.

REFERENCES

[1] S.-A.-A. Touati and D. Barthou, "On the decidability of phase ordering problem in optimizing compilation," in *Proc. ACM Computing Frontiers*, 2006.

[2] P. A. Kulkarni, D. B. Whalley, G. S. Tyson, and J. W. Davidson, "Exhaustive optimization phase order space exploration," in *Proc. IEEE/ACM CGO*, 2006.

[3] A. Haj-Ali, Q. Huang, W. Moses, J. Xiang, K. Asanovic, J. Wawrzynek, and I. Stoica, "Autophase: Juggling HLS phase orderings in random forests with deep reinforcement learning," in *Proc. MLSys*, 2020.

[4] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, "CompilerGym: Robust, performant compiler optimization environments for AI research," in *Proc. IEEE/ACM CGO*, 2022.

[5] S. Jain, Y. Andaluri, S. VenkataKeerthy, and R. Upadrasta, "POSET-RL: Phase ordering for optimizing size and execution time using reinforcement learning," in *Proc. IEEE ISPASS*, 2022.

[6] Y. Liang, K. Stone, A. Shameli, C. Cummins, M. Elhoushi, J. Guo, B. Steiner, X. Yang, P. Xie, H. Leather, and Y. Tian, "Learning compiler pass orders using coreset and normalized value prediction," in *Proc. ICML*, 2023.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[8] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.

[9] C. Cummins, Z. Fisches, T. Ben-Nun, T. Hoefler, M. O'Boyle, and H. Leather, "ProGraML: A graph-based program representation for data flow analysis and compiler optimizations," in *Proc. ICML*, 2021.

[10] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "MLGO: a machine learning guided compiler optimizations framework," *arXiv:2101.04808*, 2021.

[11] R. Mammadli, A. Jannesari, and F. Wolf, "Static neural compiler optimization via deep reinforcement learning," in *Proc. IEEE/ACM LLVM-HPC Workshop*, 2020.

[12] C. Cummins, V. Seeker, D. Grubisic, B. Rozière, J. Gehring, G. Synnaeve, and H. Leather, "LLM compiler: Foundation language models for compiler optimization," in *Proc. ACM CC*, 2025.

[13] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. IEEE WWC*, 2001.

[14] Y. Hara, H. Tomiyama, S. Honda, H. Takada, and K. Ishii, "CHStone: A benchmark program suite for practical C-based high-level synthesis," in *Proc. IEEE ISCAS*, 2008.

[15] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. ACM PLDI*, 2011.